TGU-YourTTS

Teacher-Guided Unlearning for Speaker Identity in Zero-Shot Text-to-Speech Systems

https://github.com/M22AIE243/SU_project

Team Members

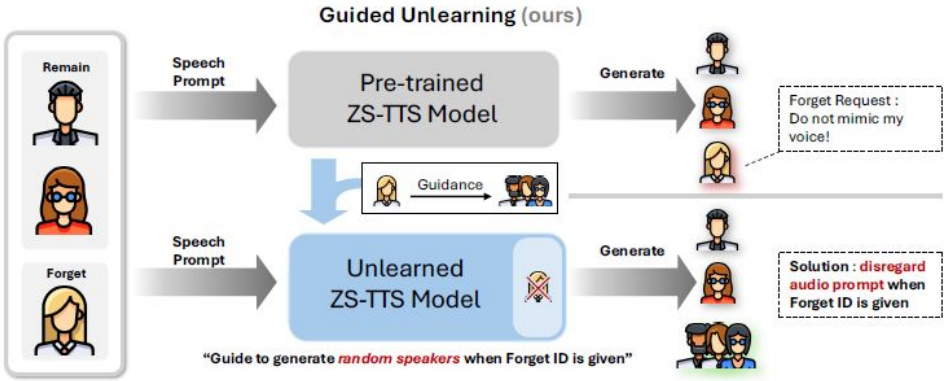
Suvendu Mullick (M22AIE218)
Roushan Kumar (M22AIE218)
Rajesh Mohan (M22AIE221)

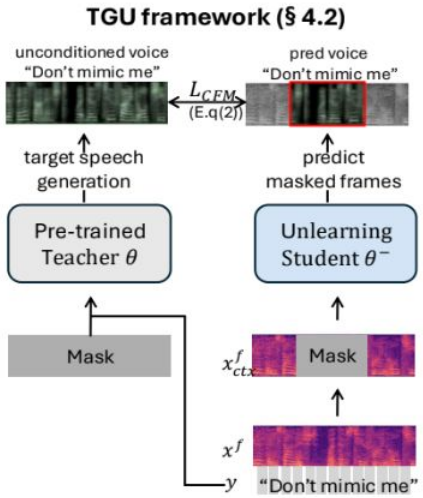
Problem statement

Given a pre-trained ZS-TTS model θ capable of replicating any speaker from a 3-second prompt, and a forget set F of speakers who have requested removal of their voice from the system, we seek an unlearned model θ^- such that:

- For remain speakers $r \in R$: $\theta^-(x_r, y) \approx \theta(x_r, y)$ [quality preserved]
- For forget speakers $f \in F$: $\theta^-(x_f, y) \approx \hat{x}_{spk \neq f}$ [identity erased, random voice]

The key challenge specific to ZS-TTS is that approximate unlearning (excluding forget speaker data from training) is insufficient — the model generalises to unseen voices via in-context learning and can still clone forget speakers at inference time.



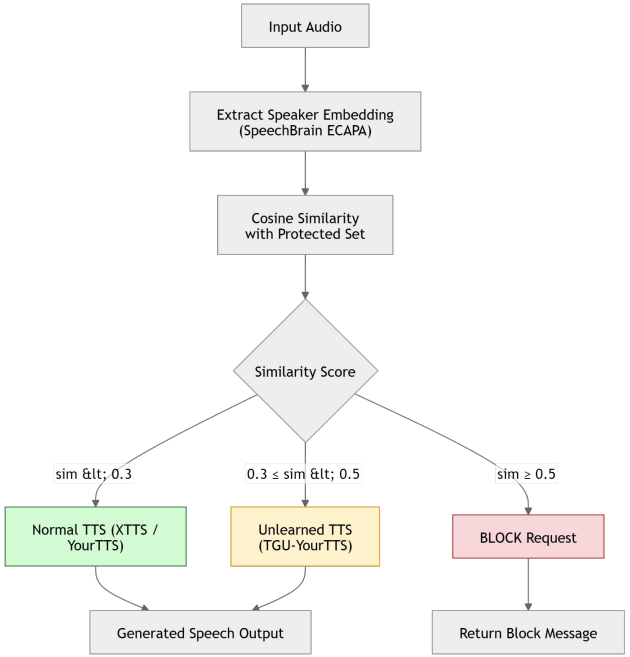


Architecture & Implemenation

- Speaker verification layer (ECAPA)
- Decision rule (cosine similarity)
- TTS generation (XTTS / YourTTS)
- Unlearning (TGU-YourTTS)

Criterion	VoiceBox (paper)	YourTTS (ours)
Source code	❌ Not released	✅ Full code on GitHub
Pre-trained weights	❌ Not available	✅ pip install TTS
Training compute	~50,000 GPU hours	Semester-feasible
Dataset	LibriHeavy (50K hrs)	VCTK (44 hrs, 109 spk)
Speaker conditioning	Masked audio context	Explicit d-vectors (512-d)
Architecture	Flow matching (CFM)	VAE + normalising flow (VITS)

Table 2: Comparison of VoiceBox vs YourTTS for TGU implementation.



- We selected YourTTS (Casanova et al., 2022, ICML) as our base model for the following reasons:
- Dataset: VCTK (Veaux et al.) contains 109 speakers recorded at 48 kHz, covering diverse English accents

Forget set (10 speakers): / Remain set (99 speakers)

Key contribution - TGU Adaptation for VITS

- **Loss function**

  VoiceBox uses Conditional Flow Matching (CFM) where the loss is a vector-field matching objective.
  VITS uses a VAE with normalising flow and HiFi-GAN vocoder. We adapt TGU's spirit

  ```
  L_forget = L1(student_wav, teacher_wav.detach()) + β · KL_posterior
  Normal VITS loss on remain batch (mel + KL + duration + adversarial)
  ```
  $L_total = \lambda \cdot L_remain + (1 - \lambda) \cdot L_forget \quad [\lambda = 0.2]$

- **Algorithmic Contribution 1: Adaptive Lambda Scheduling**

  The reference paper uses a fixed λ=0.2 throughout training (Eq. 9). We introduce an adaptive schedule that anneals λ from 0.5 → 0.2 over a warmup period. At the start of unlearning, the forget speakers' identity is strongly encoded in model weights. A lower λ (higher weight on L_forget) applies more aggressive unlearning pressure at the beginning when identity erasure matters most. As the model converges, λ increases toward 0.2 to emphasise quality preservation for remain speakers.

  $\lambda(t) = \lambda_start + (t / warmup_steps) \times (\lambda_final - \lambda_start)$

- **Algorithmic Contribution 2: Speaker Similarity Regulariser**

  We add an explicit penalty when the unlearned model's output is still too similar to the forget speaker:
  `L_sim_reg = weight × ReLU(CosineSIM(gen_emb, forget_emb) − τ)`

Results - TGU unlearning

```
============================================================
  TGU Training — 10000 steps
  Forget speakers : ['p225', 'p226', 'p227', 'p228', 'p229', 'p230', 'p231', 'p232', 'p233', 'p234']
  Output dir      : /scratch/data/m22aie221/workspace/Project_Speech-Understanding/outputs/
  λ               : adaptive 0.5→0.2
============================================================


TGU Unlearning:   0%|          | 0/10000 [00:00<?, ?it/s]/iitjhome/m22aie221/miniconda3/envs/su_unlearn/lib/python3.10/site-p
  warnings.warn("Detected call of `lr_scheduler.step()` before `optimizer.step()`. "

TGU Unlearning:   0%|          | 1/10000 [00:02<7:22:04,  2.65s/it]
TGU Unlearning:   0%|          | 9/10000 [00:02<37:49,  4.40it/s]
TGU Unlearning:   0%|          | 13/10000 [00:02<26:06,  6.37it/s]
TGU Unlearning:   0%|          | 17/10000 [00:03<20:09,  8.26it/s]
TGU Unlearning:   0%|          | 22/10000 [00:03<15:45, 10.55it/s]
TGU Unlearning:   0%|          | 26/10000 [00:03<13:37, 12.20it/s]
```

```
TGU Unlearning: 100%|██████████| 9993/10000 [04:44<00:00, 33.95it/s]
TGU Unlearning: 100%|██████████| 10000/10000 [04:44<00:00, 23.71it/s]
TGU Unlearning: 100%|██████████| 10000/10000 [04:44<00:00, 35.12it/s]
Step  8500 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=6.97e-06
Step  8550 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=6.58e-06
Step  8600 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=6.21e-06
Step  8650 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=5.85e-06
Step  8700 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=5.50e-06
Step  8750 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=5.17e-06
Step  8800 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=4.85e-06
Step  8850 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=4.54e-06
Step  8900 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=4.24e-06
Step  8950 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=3.95e-06
Step  9000 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=3.68e-06
[YourTTSWrapper] Saved to /scratch/data/m22aie221/workspace/Project_Speech-Understanding/outputs/tgu_step_9000.pth
Step  9050 | λ=0.200 | L_forget=0.0000 | L_remain=0.0000 | L_total=0.0000 | LR=3.42e-06
```

Results & Challenges

During training TGU with VITS we faced challenges. Key Issues Faced :-

- Inconsistent APIs across Coqui layers
  - `tts()`, `forward()`, `inference()` all differ
- Tokenizer not accessible or version-dependent
- Frequent runtime errors:
  - argument mismatch
  - shape mismatch (e.g., 192 vs 196 channels)
- Training instability:
  - Loss collapsing to **0.0000**
- Inference failures:
  - `unexpected keyword argument 'text'`

Although the training pipeline executed successfully, the model failed to learn meaningful unlearning behavior due to architectural and API limitations of VITS, resulting in zero-loss convergence and unreliable inference.”

References

[1] T. Kim*, J. Kim* et al., "Do Not Mimic My Voice: Speaker Identity Unlearning for Zero-Shot Text-to-Speech," Proc. 42nd ICML, PMLR 267, 2025. arXiv:2507.20140

[2] E. Casanova et al., "YourTTS: Towards Zero-Shot Multi-Speaker TTS and Zero-Shot Voice Conversion for Everyone," Proc. ICML, pp. 2709–2720, 2022.

Appendix A - Hyperparamters

```yaml
# configs/tgu_vctk.yaml (key parameters)
tgu:
  lambda_weight:       0.2      # paper value
  adaptive_lambda:     true     # our addition
  lambda_start:        0.5      # anneal from
  lambda_warmup_steps: 2000     # steps to reach 0.2
  unlearn_steps:       10000    # total training steps
  batch_size:          8        # GPU memory constraint
  learning_rate:       1.0e-4   # matching paper
  warmup_steps:        500      # LR warmup
  grad_clip:           5.0      # prevents instability
  forget_sample_prob:  0.2      # paper Appendix B
  sim_regularization:  true     # our addition
  sim_threshold:       0.3      # cosine SIM threshold
  sim_penalty_weight:  0.5      # regulariser weight
```

Appendix - B

```
Project_Speech-Understanding/
├── configs/tgu_vctk.yaml         ← all hyperparameters and paths
├── data/vctk_dataset.py           ← VCTK loader with 3-tier d-vector lookup
├── models/yourtts_wrapper.py      ← YourTTS wrapper (8 bugs fixed)
├── unlearning/
│   ├── tgu_trainer.py             ← TGU Eq.7,8,9 + adaptive λ + SIM reg.
│   └── baselines.py               ← FT, NG, KL baselines
├── evaluation/
│   ├── eval_spkzrf.py             ← spk-ZRF metric (Eq. 10,11,12)
│   └── eval_metrics.py            ← WER (Whisper) + SIM (WavLM)
├── analysis/manifold_analysis.py  ← 5-level real manifold analysis
├── scripts/
│   ├── 00_setup.sh                ← dependency installation
│   ├── 01_compute_embeddings.sh   ← d-vector extraction (model_path fixed)
│   ├── 03_run_tgu.sh              ← train all methods
│   └── 04_evaluate.sh             ← full evaluation pipeline
├── jobs/                          ← SLURM job scripts for IIT cluster
├── train_tgu.py                  ← main entry point
├── inference.py                  ← audio generation
├── evaluate.py                   ← Table 1 format results
└── debug_tgu_pipeline.py          ← component validation script (fixed)
```